Experiment No.: 06
Title: K-means clustering algorithm

Batch: B3 **Roll No: 16014225801** **Experiment No: 06**

Aim: To implement any four K-value selection algorithms along with K means clustering algorithm.

Resources needed: Any programming language.

Pre Requisite: Machine learning concepts, Statistics

Theory:

Introduction

K-Means is one of the most widely used unsupervised machine learning algorithms for clustering. Its primary goal is to partition a dataset into K distinct, non-overlapping groups (clusters) based on feature similarity. It is distance-based, meaning it groups data points such that those within the same cluster are more similar to each other than to points in other clusters. K-Means is popular for its simplicity, scalability, and efficiency, and is often used in applications such as image segmentation, customer segmentation, anomaly detection, and document clustering.

Objective: Minimize the Within-Cluster Sum of Squares (WCSS), which measures the variance within each cluster:

$$WCSS = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$$

where:

- K = number of clusters
- C_i = set of points belonging to cluster i
- μ_i = centroid of cluster i

K-Value Selection Algorithms

Choosing the right K is crucial for effective clustering. Common methods include:

1. Elbow Method:
   - Plot WCSS against different K values.
   - Select K at the "elbow" point where the rate of WCSS decrease sharply changes.
2. Silhouette Analysis:
   - Measures how similar a point is to its own cluster compared to others.
   - Silhouette score ranges from -1 (poor clustering) to 1 (dense and well-separated clusters).
   - Select K that maximizes the silhouette score.
3. Gap Statistic:
   - Compares WCSS for actual data with WCSS from uniformly distributed random data.
   - Select K with the largest gap.
4. Information Criterion Approaches:
   - Use statistical measures like following for optimal K selection.
     - AIC (Akaike Information Criterion)
     - BIC (Bayesian Information Criterion)

Procedure:

1. Download the unsupervised dataset.
2. Implement any two K value selection algorithms.
3. Using the selected value of K from the selection algorithms, implement K K-means clustering algorithm.
4. Compare the results with different values of k and comment on same.

Results:

Code

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

path = "/Users/bhumit/Desktop/Sem 5/india_2000_2024_daily_weather.csv"
data = pd.read_csv(path)

features = [
    "temperature_2m_max",
    "temperature_2m_min",
    "apparent_temperature_max",
    "apparent_temperature_min",
    "precipitation_sum",
    "rain_sum",
    "wind_speed_10m_max",
    "wind_gusts_10m_max"
]

X = data[features].dropna().values

print("Dataset shape:", X.shape)
```

Output

```
Dataset shape: (91320, 8)
```

Code

```
mean = np.mean(X, axis=0)
std = np.std(X, axis=0)

X = (X - mean) / std

# K-MEANS ALGORITHM
def kmeans(X, k, max_iterations=100):

    # Randomly select initial centroids
    random_indices = np.random.choice(len(X), k, replace=False)
    centroids = X[random_indices]

    for iteration in range(max_iterations):

        # Calculate distance of every point from every centroid
        distances = np.zeros((len(X), k))

        for i in range(k):
            distances[:, i] = np.sqrt(
                np.sum((X - centroids[i]) ** 2, axis=1)
            )
```

```
        # Assign each point to nearest centroid
        labels = np.argmin(distances, axis=1)

        # Calculate new centroids
        new_centroids = np.zeros((k, X.shape[1]))

        for i in range(k):
            points = X[labels == i]

            if len(points) > 0:
                new_centroids[i] = np.mean(points, axis=0)
            else:
                new_centroids[i] = centroids[i]

        # Check convergence
        if np.allclose(centroids, new_centroids):
            break

        centroids = new_centroids

    return labels, centroids

# CALCULATE WITHIN CLUSTER SUM OF SQUARES
def calculate_wcss(X, labels, centroids):

    wcss = 0

    for i in range(len(centroids)):

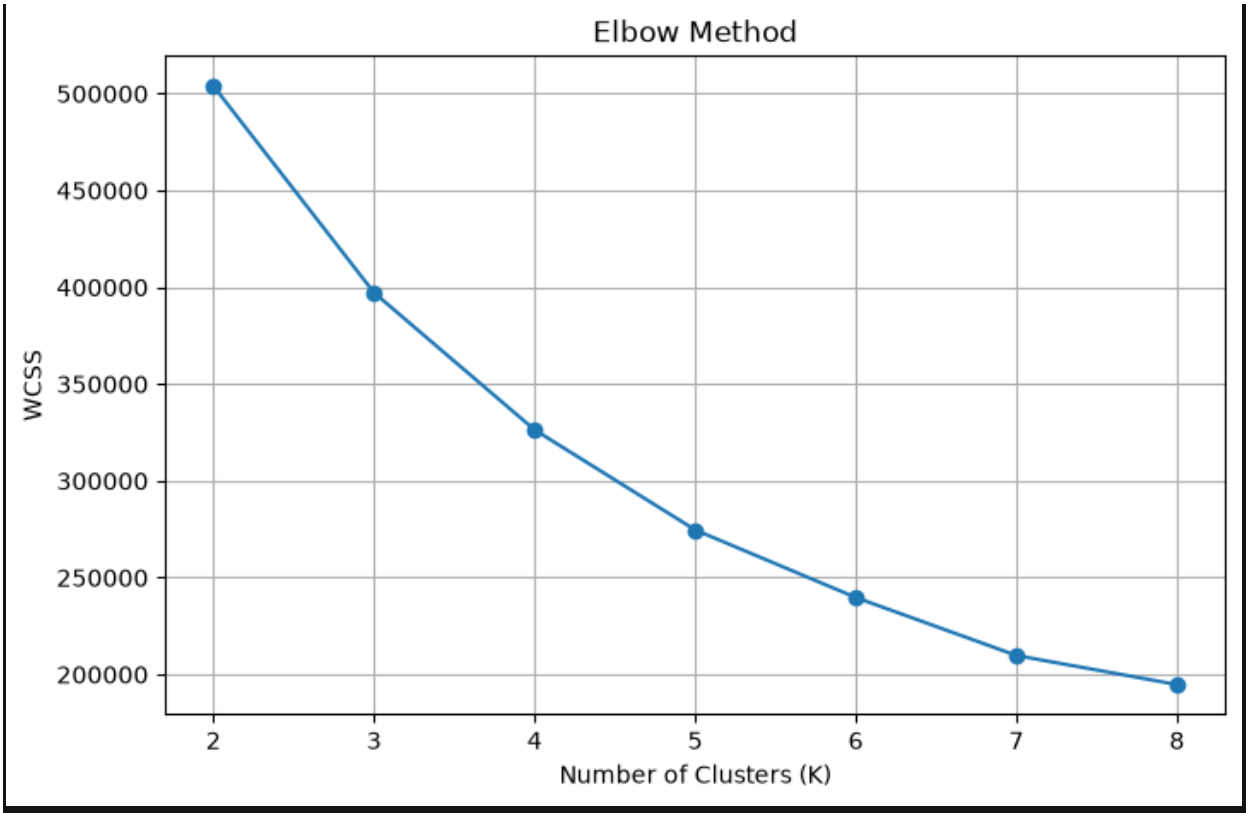
        cluster_points = X[labels == i]

        if len(cluster_points) > 0:
            wcss += np.sum(
                (cluster_points - centroids[i]) ** 2
            )

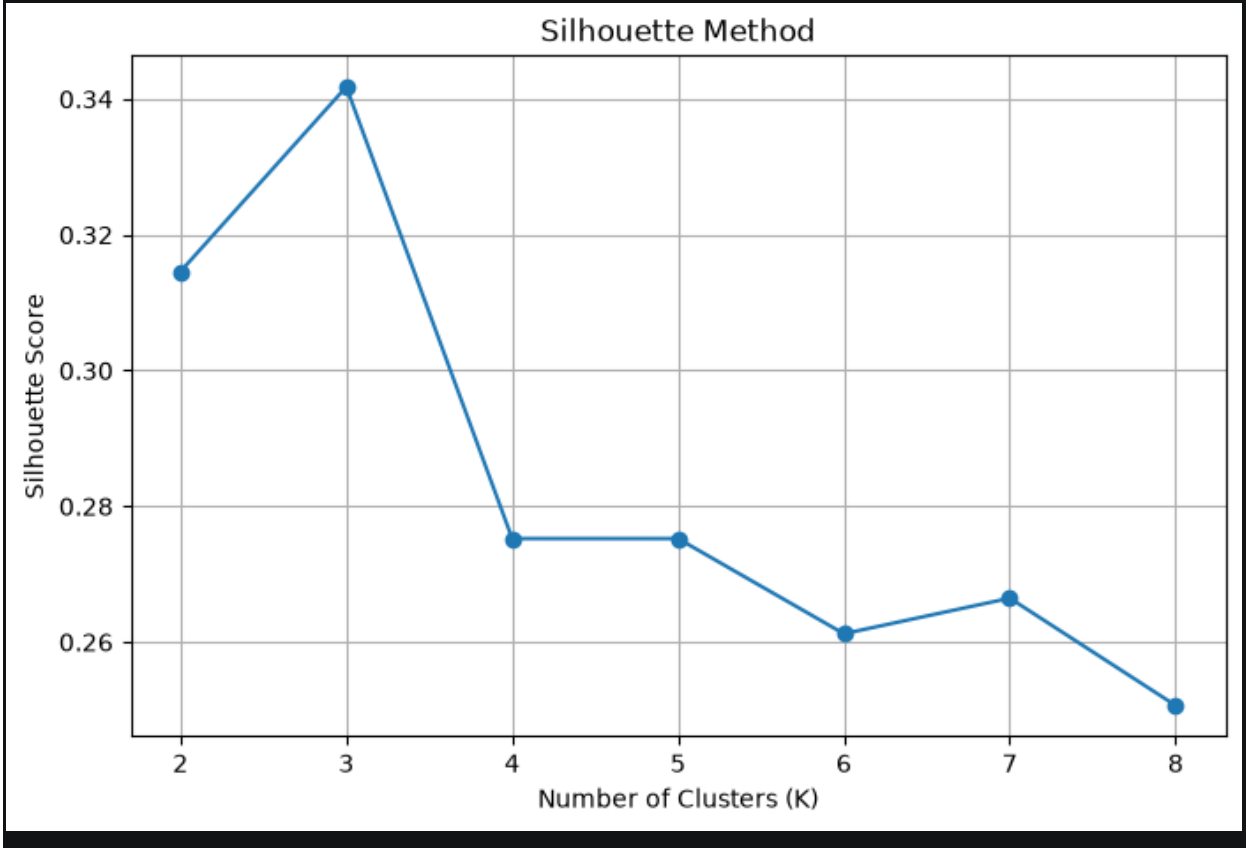
    return wcss
```

Output

```
K = 2  WCSS = 503931.75907471264
K = 3  WCSS = 397216.03270009137
K = 4  WCSS = 326351.99134990404
K = 5  WCSS = 274647.4944322943
K = 6  WCSS = 239746.23061533459
K = 7  WCSS = 209795.33345723033
K = 8  WCSS = 194758.55675927293
```




```
K = 2  Silhouette Score = 0.3145752764469564
K = 3  Silhouette Score = 0.34179367023019075
K = 4  Silhouette Score = 0.27524108806546743
K = 5  Silhouette Score = 0.27523607943374645
K = 6  Silhouette Score = 0.2612187618816277
K = 7  Silhouette Score = 0.2664594793129798
K = 8  Silhouette Score = 0.25064733624344027
```



Code

```
# 4. SELECT BEST K
best_k = k_values[np.argmax(silhouette_values)]

print("\nBest K according to Silhouette Method =", best_k)

# 5. FINAL K-MEANS CLUSTERING
labels, centroids = kmeans(X, best_k)
data_clean = data[features].dropna().copy()
data_clean["Cluster"] = labels

print("\nFinal Cluster Distribution:")
print(data_clean["Cluster"].value_counts().sort_index())
```

Output

```
Best K according to Silhouette Method = 3

Final Cluster Distribution:
Cluster
0     3450
1    55293
2    32577
Name: count, dtype: int64
```

Code

```
# 6. COMPARE DIFFERENT VALUES OF K
print("\nComparison of different K values:")
for k in range(2, 7):
    labels_temp, centroids_temp = kmeans(X, k)
    wcss = calculate_wcss(
        X,
        labels_temp,
        centroids_temp
    )

    silhouette = silhouette_score(
        X,
        labels_temp
    )

    print(
        "K =", k,
        "| WCSS =", round(wcss, 2),
        "| Silhouette =", round(silhouette, 4)
    )
```

Output

```
Comparison of different K values:
K = 2 | WCSS = 503931.76 | Silhouette = 0.3165
K = 3 | WCSS = 397216.03 | Silhouette = 0.3432
K = 4 | WCSS = 326351.73 | Silhouette = 0.2724
K = 5 | WCSS = 274647.37 | Silhouette = 0.2772
K = 6 | WCSS = 239746.23 | Silhouette = 0.2612
```

Questions:

1. What is the main difference between K-means and K-nearest neighbours?
2. What are some stopping criteria for k-means clustering?

Answers:

1. K-means is an unsupervised learning algorithm used for clustering. It divides unlabelled data into K groups (clusters) based on similarity between data points.
K-nearest neighbours (KNN) is a supervised learning algorithm used for classification or regression. It uses already labelled data and predicts the class/value of a new data point based on its K nearest neighbours.

2. K-means can stop when any of the following conditions are satisfied:

- Centroids do not change: The positions of the cluster centres remain the same or change by a very small amount.
- Cluster assignments do not change: Data points remain assigned to the same clusters after an iteration.
- Error/WCSS change is very small: The improvement in the within-cluster sum of squares becomes negligible.
- Maximum iterations reached: The algorithm stops after a predefined number of iterations, such as 100 or 1000, even if complete convergence has not occurred.

Conclusion:

K-means successfully grouped similar weather data, while Elbow and Silhouette methods helped select an appropriate number of clusters.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

Websites:

1. http://www.vmware.com/in
2. http://www.xenproject.org